Выбор политики маршрутизации в двухмодельных каскадах: влияние данных, структуры ошибок и вычислительного бюджета

Anastasia Shlopak
Научный руководитель: Archil Maysuradze, PhD

Аннотация

Каскадные системы позволяют адаптивно распределять вычислительные ресурсы, передавая часть запросов от дешёвого предиктора более дорогому. Эффективность такой системы зависит от способности маршрутизатора оценить ожидаемое изменение качества ответа: дорогая модель может как исправить ошибку, так и заменить правильный ответ неправильным. Существующие маршрутизаторы используют различные обучающие цели и параметризации, однако для выбора того или иного подхода для новой задачи важно учитывать доступные данные, структуру ошибок моделей и ограничения эксплуатации. Цель исследования заключается в выявлении условий, при которых различные способы обучения маршрутизатора будут обеспечивать наилучшее соотношение качества и вычислительных затрат. Основная задача — разработать систему рекомендаций по выбору маршрутизатора на основе характеристик данных и используемых моделей. В работе рассматривается каскад из двух фиксированных классификаторов и маршрутизатора, получающего на вход признаки объекта и результат дешёвой модели. Для сравнения будут выбраны такие политики маршрутизации, как прямая регрессия прироста качества, разность оценок правильности моделей, оценивание вероятностей исправления и ухудшения качества ответа, неуверенность только дешёвой модели, сложность признаков входного объекта. Переносимость сформированных рекомендаций планируется проверить на новых наборах данных и парах классификаторов, не использованных при выявлении закономерностей и выборе параметров методов. Итогом работы должны стать экспериментально обоснованные правила выбора способа обучения маршрутизатора, позволяющие определить, когда дополнительная сложность оправданна, а когда сопоставимое качество достигается более простым решением.

Ключевые слова: каскад классификаторов, маршрутизация, ожидаемая полезность, парная разметка, совместные ошибки, выбор метода обучения.

1. Введение

Каскадная обработка позволяет выделять дополнительное вычисление запросам, на которых оно полезно. Первая модель отвечает всегда, после чего маршрутизатор сохраняет её ответ либо передаёт запрос второму эксперту. Поскольку второй эксперт также ошибается, более высокая средняя accuracy не гарантирует полезность каждого перехода. Цель исследования — разработать экспериментально обоснованные рекомендации по выбору способа обучения маршрутизатора для заданных данных, пары моделей и бюджета.

Объект исследования — двухмодельный каскад с фиксированными классификаторами и обучаемым правилом эскалации. На обучающих примерах известна правильность обеих моделей; при эксплуатации роутер использует только запрос и доступный результат первой

модели. Это позволяет непосредственно наблюдать спасения ошибок и вредные замены и сравнивать разные способы оценивания ожидаемой пользы одного и того же перехода.

Сравнительная полезность эскалации формализована в литературе. Jitkrittum et al. [1] рассматривают разность вероятностей правильности и прямую регрессию signed gain, а также условия достаточности confidence-правила. Post-hoc обучение связано с передачей решения ошибающемуся эксперту [2]; для языковых моделей изучены токенные признаки [3] и выбор продолжения по ожидаемой полезности [4]. Эти работы задают целевую величину и сильные альтернативные методы для сравнения.

Остаётся практический вопрос выбора оценивателя при конечном объёме парных данных. Источник и состав обучения по-разному влияют на роутеры [5, 6]. В uplift modeling показаны искажение оценки эффекта после пересэмплирования и роль редких событий [7]. Таким образом, сравнение должно учитывать одновременно поддержку rescue/harm, смещение отбора, доступную информацию и квоту, а не только средний разрыв точности экспертов.

Предлагается фиксировать экспертов и признаки, изменять объём и состав именно обучения роутера и оценивать методы при одинаковой квоте. Задачи исследования — выявить режимы предпочтительности методов, объяснить различия и проверить переносимость рекомендаций на новые задачи и пары моделей. Дополнительно к accuracy рассматриваются исправленные ошибки, вредные замены, упущенные спасения и предотвращённый вред: близкая итоговая точность может скрывать разные профили решений.

Первый эксперимент проводится на BANKING77 с предвычисленными прогнозами независимо обученных классификаторов. Единый кэш, совпадающие подвыборки и ограничения на доступную информацию обеспечивают сопоставимость внутри условия. Базовое сравнение включает confidence, weak-error, direct и rescue/harm-головы, кривую ошибки и таблицу решений. Отдельная проверка статистической значимости компонентов и независимая оценка переносимости рекомендаций остаются следующими проверками.

2. Обзор литературы

Исследования маршрутизации рассматривают, какую часть вычислений достаточно выполнить для конкретного запроса и когда дополнительный эксперт способен улучшить ответ. Для нашей задачи существенны три группы подходов: правила на основе уверенности первой модели, сравнительное оценивание полезности перехода и исследования зависимости обучения/выбора оценивателя от данных. Такое разделение позволяет сопоставить используемую информацию, обучающие цели и ограничения.

2.1. Уверенность и сравнительная полезность

Простые confidence-правила не требуют отдельного обучения роутера, но оценивают прежде всего надёжность первой модели. В работе Jitkrittum et al. [1] показано, когда этого достаточно, и разобраны случаи специализации второго эксперта, шума и сдвига распределения. Там же используется Diff-01 — регрессия разности правильности двух моделей. Поэтому преимущество сравнительного подхода связано не с самим наличием второго эксперта, а с тем, содержат ли доступные признаки информацию о неоднородности его пользы.

Для генеративных моделей уверенность дополнительно зависит от способа агрегации токенных вероятностей и длины ответа. Gupta et al. [3] показывают ограничения суммы/среднего и исследуют квантили и embeddings. Для accuracy-задач используется rescue-only цель, а для перевода — регрессия разности BLEURT. Эти цели и информационные режимы нельзя отождествлять. Добавление признаков второй модели также меняет стоимость решения.

Post-hoc обучение передачи решения учитывает ошибки и стоимость несовершенного эксперта [2]. В AutoMix ожидаемая полезность продолжения оценивается по шумному сигналу самопроверки в последовательной модели принятия решений [4]. Это более общий механизм каскада, но самопроверка требует учёта дополнительных затрат. В unified routing/cascading изучаются условия полезности разных схем в зависимости от качества

оценок [8]. Часть анализа использует искусственный шум истинных quality signals; это отличается от изменения объёма и состава реального обучения конкретного оценивателя.

2.2. Обучающие данные и оценивание разности

В RouteLLM сравнены роутеры на preference data и её дополнениях из gold- и judge-разметки [5]. Результаты демонстрируют роль соответствия источника обучения задаче. Работа Routing with Generated Data сравнивает query-answer и query-only методы при разном качестве генератора и фильтрует запросы по согласию сильных и расхождению слабых моделей [6]. Таким образом, целевой отбор информативных примеров уже является предметом исследования. В этих работах выбирается эксперт до получения первого ответа; наш базовый контроль использует post-answer признаки фиксированной первой модели.

В uplift regression сравниваются прямая оценка эффекта и разность отдельных моделей [9], а исследования meta-learners связывают предпочтительность метода со структурой эффекта и объёмом данных [10]. Обычно на объекте известен только один потенциальный исход. При нашей парной разметке оба ответа наблюдаются, поэтому аргументы об эффективности unpaired оценивателей нельзя переносить буквально. Например, на одном X с одинаковой ridge-регуляризацией регрессия $S-W$ тождественна разности двух ridge-прогнозов; различия с logistic-головами возникают из-за класса функций, потерь и регуляризации.

Особенно близка работа Nyberg and Klami [7]: сравниваются sampling schemes, оцениватели и размеры данных, разбираются искажение ранжирования и коррекция вероятностей. Несмотря на пользу большего числа редких событий, изменение состава train может менять саму оцениваемую условную величину. Это объяснение необходимо проверять отдельно от статистической поддержки rescue/harm.

Таблица 1: Подходы и ограничения, существенные для выбора способа обучения каскадного маршрутизатора.

Подход	Сильная сторона	Ограничение для нашей задачи
Confidence [1]	Простое правило без отдельной fit-процедуры	Не моделирует неоднородность пользы второго эксперта
Post-hoc gain [1, 3]	Использует сравнительную разметку и доступные признаки	Зависит от объёма данных, представления и выбранной цели
Последовательная utility [4, 8]	Явно сопоставляет качество действий и затраты	Надёжность quality/verifier signal и его цена влияют на результат
Данные для routing [5, 6]	Проверяет источники supervision и информативный отбор	Single-shot режим и разметка отличаются от парного post-answer каскада
Sampling effect estimation [7]	Разделяет дисбаланс, искажение разности и коррекцию	Непарные исходы; прямой отбор индивидуальных rescue/harm недоступен

2.3. Выбор метода и границы текущего исследования

LLMRouterBench показывает близкие результаты ведущих роутеров и конкурентоспособность простых решений при едином протоколе [11]. Эти наблюдения мотивируют проверять, оправданно ли усложнение, но близкие средние сами по себе не доказывают эквивалентность. Отдельно следует оценивать качество выбора метода: такой подход развит для CATE estimator selection [12]. В нашей парной постановке качество политики на dev можно оценить напрямую, без восстановления ненаблюдаемого контрфактического исхода.

Таким образом, существующая литература задаёт целевую полезность, сильные базовые методы и известные механизмы чувствительности к данным. Наше исследование продолжает эту линию контролируемым сравнением способов обучения при фиксированных экспертах и признаках, разном составе парных примеров и одинаковой квоте. Требуется установить, какие различия устойчивы, чем они объясняются и позволяют ли характеристики train/dev рекомендовать метод на новой задаче. Таблицы спасений/вреда используются для анализа механизмов, а не как самостоятельное доказательство новизны или превосходства.

3. Постановка задачи

3.1. Данные, предположения и ограничения

Даны запросы и эталоны $(x, y) \sim P$, два фиксированных классификатора M_w, M_s и выборка парных исходов их работы. Обозначим $W = \mathbf{1}\{M_w(x) \text{ правилен}\}$, $S = \mathbf{1}\{M_s(x) \text{ правилен}\}$ и $D = S - W \in \{-1, 0, 1\}$. Здесь $D = 1$ — спасение, $D = -1$ — вредная замена, $D = 0$ — одинаковая правильность ответов. Предположение о независимости ошибок моделей не требуется; большая accuracy второй модели не постулируется.

Сначала вызывается M_w. Роутеру доступны признаки $z = \phi(x, M_w(x))$, которые можно получить до второго вызова. Текущий ответ M_s и эталон при решении неизвестны. При эскалации второй ответ заменяет первый без отдельного судьи или объединения ответов. Правильность обеих моделей используется только как обучающая разметка или для оценки качества.

Эксперты обучаются на отдельной части данных и затем фиксируются. Router-fit, policy-dev и test не пересекаются по объектам с этой частью или друг с другом. Все признаки преобразуются по параметрам, оценённым только на router-fit. В базовом опыте train/dev/test роутера представляют естественный поток. При целевом отборе router-train его распределение может отличаться от P; поправка на такой отбор должна учитываться отдельно. Новые запросы рассматриваются как выборка из целевого потока, а возможная семантическая зависимость запросов ограничивает перенос обычных IID-оценок неопределённости.

3.2. Полезность, стоимость и внешние критерии

Условная полезность эскалации:

$$\begin{aligned} g(z) &= \mathbb{E}[D \mid z] \\ &= P(W = 0, S = 1 \mid z) - P(W = 1, S = 0 \mid z) \\ &= P(S = 1 \mid z) - P(W = 1 \mid z). \end{aligned} \tag{1}$$

Для решения $r(z) \in \{0, 1\}$ внешний критерий — accuracy каскада:

$$Q(r) = \mathbb{E}[W + r(z)D], \qquad \mathbb{E}[c_w + c_{\phi,r} + r(z)c_s] \le B. \tag{2}$$

Здесь c_w — первый вызов, $c_{\phi,r}$ — признаки и роутер, c_s — дополнительный второй вызов. В базовом опыте вместо измеренного денежного бюджета используется одинаковая для всех методов квота: ровно $k = \lfloor bN \rfloor$ вызовов на пакет из N запросов. Выбираются top - k оценок с общим разрешением равных scores. Это пакетный отбор, а не проверенный порог для online-потока.

Профиль принятого решения относительно противоположного действия описывается $Z_r = (2r - 1)D$ и таблицей

$$T_{v,a} = \sum_{i=1}^{N} \mathbf{1}\{Z_{r,i} = v, r_i = a\}, \qquad v \in \{-1, 0, 1\}, \quad a \in \{0, 1\}. \tag{3}$$

При эскалации строка $+1$ означает спасение, -1 — вред; без эскалации $+1$ означает предотвращённый вред, -1 — упущенное спасение. Нейтральный исход объединяет два правильных и два ошибочных ответа. При числе спасений R и вредных замен H эмпирический прирост accuracy равен $(R - H)/N$. Поэтому таблица раскрывает компоненты агрегированной метрики, но не заменяет проверку их различий.

3.3. Внутренний критерий и процедура выбора

Для каждого способа обучения параметры оптимизируются на router-fit, сила регуляризации выбирается по MSE наблюдаемого D на policy-dev. Критерий настройки отделён от внешней accuracy при фиксированном b. Для вероятностных голов используется log-loss; отсутствующий класс предсказывается с нулевой вероятностью, единственный — константой.

В прямом варианте ищется линейная функция $f_\theta(z)$; intercept не штрафуется. Внутренняя задача:

$$\widehat{\theta}_\lambda \in \arg\min_\theta \left\{ \frac{1}{n} \sum_{i=1}^{n} (D_i - f_\theta(z_i))^2 + \lambda \|\theta_{\text{coef}}\|_2^2 \right\}. \tag{4}$$

Основная исследовательская задача — выбор способа обучения $a \in \mathcal{A}$ по разрешённым характеристикам данных, пары и бюджета. Для конфигурации τ обозначим обученный роутер $r_{\tau,a}$, его качество $Q_\tau(r_{\tau,a})$ и descriptors d_τ. Рекомендация $\pi(d_\tau)$ не использует test; на новой задаче допускаются обучение и настройка роутера на её разрешённых train/dev. Контроль переноса требует целиком отложенных задач или пар. При $a_\tau^* \in \arg\max_{a \in \mathcal{A}} Q_\tau(r_{\tau,a})$ задача выбора записывается как

$$\pi^* \in \arg\min_{\pi \in \Pi} \mathbb{E}_\tau \left[Q_\tau(r_{\tau,a_\tau^*}) - Q_\tau(r_{\tau,\pi(d_\tau)}) \right] \tag{5}$$

при сопоставимом бюджете парной разметки, включая настройку.

4. Вычислительные эксперименты

4.1. Цель и проверяемые свойства

Цель базового эксперимента — сравнить ошибку и профили решений маршрутизаторов при фиксированных экспертах, доступных признаках и одинаковой квоте эскалаций. Проверяется зависимость от количества парных примеров обучения и бюджета второго вызова. Прирост каскада над первой моделью, разница между правилами при одной квоте и обмен спасений на вред рассматриваются как разные результаты.

4.2. Данные и разбиение

Используется BANKING77: англоязычные банковские обращения с 77 намерениями [13]. Источник — PolyAI revision `57ec275d8078af65b7731c2a98be812d844a6d6b`, лицензия CC BY 4.0. Из 10 003 train-строк удалены четыре нормализованных повтора; 9 999 строк стратифицированно разделены на expert-train 5 999, calibration 1 000, router-fit 2 000 и policy-dev 1 000. Нормализация для аудита — NFKC, нижний регистр и свёртка пробелов. Между train-частями текстовых пересечений нет. Official test 3 080 сохранён; дополнительная оценка без семи train/test пересечений и одного test-повтора содержит 3 072 строки. Manifest фиксирует исходные индексы, SHA и seeds разбиения 20260922, 20260923, 20260924. Часть calibration используется для аудита экспертов, не для настройки роутера.

4.3. Фиксированные эксперты

В базовой конфигурации используются две пары: TF–IDF+LR с BERT-tiny и TF–IDF+LR с frozen MiniLM+LR. Все эксперты обучаются только на expert-train. Внутреннее разбиение 80/20 служит для выбора параметров, после чего выполняется refit на всех 5 999 строках.

TF–IDF использует word n-grams (1,2), min_df=2, sublinear TF и ограничение 30 000 признаков. LR настраивается по $C \in \{0.25, 1, 4, 16\}$; выбран $C = 16$. Frozen MiniLM — `sentence-transformers/all-MiniLM-L6-v2`, revision `1110a243fdf4706b3f48f1d95db1a4f5529b4d41`: нормализованный mean pooling и LR, $C \in \{0.25, 1, 4, 16, 64\}$, выбран $C = 64$. BERT-tiny — `prajjwal1/bert-tiny`, revision `6f75de8b60a9f8a2fdf7b69cbd86d9e64bcb3837`; mean pooling и линейная классификационная голова. Во внутренней валидации выбраны восемь эпох; refit выполнен с AdamW, batch 32, encoder LR 10^{-4}, head LR 10^{-3}, weight decay 0.01, warmup/linear decay и gradient norm cap 1. Максимальная длина — 64 токена, seed обучения — 20261002.

Accuracy на official test: TF–IDF 86.56%, BERT-tiny 84.71%, frozen MiniLM 92.31%. Направления исходных пар заданы измеренной стоимостью вызова, а не точностью: более дорогой эксперт может быть хуже в среднем. Прогнозы экспертов зафиксированы до обучения роутеров.

4.4. Признаки, методы и настройка

Восемь признаков: максимальная вероятность, top-two margin, нормированная энтропия, сумма квадратов вероятностей, вторая и третья вероятности и log1p длины запроса в символах/словах. Ответ и признаки второй модели недоступны при решении. Правильность и эталон используются как supervision и для оценки, не как входные признаки.

Сравниваются confidence (низкая максимальная вероятность), logistic weak-error, ridge direct на D и две logistic rescue/harm-головы со score $\widehat{p}_+ - \widehat{p}_-$. Weak-error вычитает из предсказанной ошибки первой модели среднюю train-ошибку второй; это proxy, а не точная условная сравнительная полезность. В каждом условии методы используют одинаковые fit/dev строки, признаки и tie-breaking keys. Классы функций и число параметров различаются; сравнение не является строго capacity-matched.

Объёмы fit — 200/500/1 000, dev=fit/2; 10 seeds подвыборок 0–9, квоты 10/20/40%. Подвыборки без возвращения вложены по размеру; scaler оценивается только на выбранном fit. L2-grid — $\{0.001, 0.01, 0.1, 1\}$; выбор — по dev MSE, при точном равенстве выбирается более сильная регуляризация. Для ridge $\alpha = n\lambda$, для logistic $C = (n\lambda)^{-1}$; intercept не штрафуется. Train/dev/test роутера сохраняют естественное распределение. Бюджет данных включает настройку: 300/750/1 500 использованных пар.

Вычисляются accuracy, observed-D MSE, числа спасений и вредных замен, таблица Z_r и разброс по обучениям. Для каждого пакета выбираются ровно $\lfloor bN \rfloor$ запросов; при 20% это 616 вызовов из 3 080. Квота не является измеренной экономией денег или end-to-end latency. Базовая сетка содержит 60 условий и 180 финальных обучаемых роутеров; с четырьмя правилами, тремя квотами и двумя версиями test получается 1 440 строк метрик.

4.5. Результаты

Таблица 2: TF–IDF → frozen MiniLM, fit 1 000/dev 500, 20% эскалаций. Средние и SD по 10 подвыборкам.

Способ	Accuracy, %	SD, п.п.
confidence	90,68	0,00
weak_error	91,27	0,07
direct	91,48	0,23
events	91,53	0,17

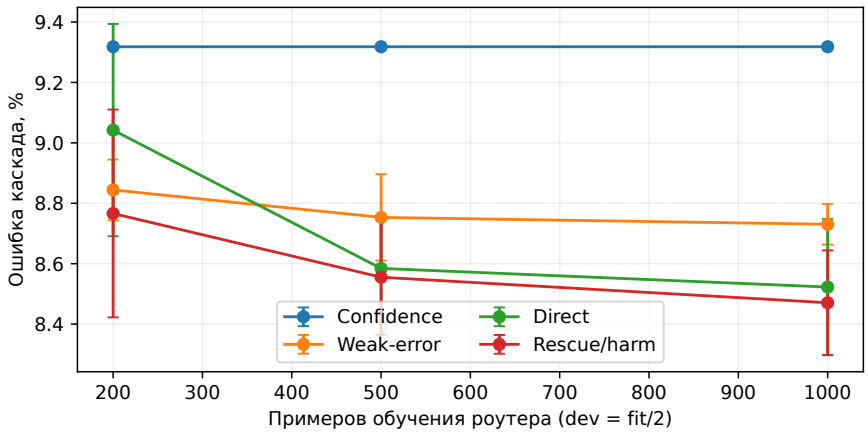


Рис. 1: Ошибка каскада при изменении fit и dev=fit/2, TF–IDF → frozen MiniLM, 20% вызовов. Полосы — SD по 10 подвыборкам обучения.

Результаты представлены в таблице 2 и на рисунке 1. При увеличении обучения ошибка части правил снижается; SD по подвыборкам не является доверительным интервалом из

независимых задач. При fit 1 000 direct и events имеют близкие средние accuracy, поэтому различия в сотых процента не рассматриваются как практическое преимущество.

На рисунке 2 показан описательный пример другой пары. Confidence исправляет в среднем 123 ответа и портит 102, Events — 85.2 и 51.1. Таким образом, меняется профиль решений: снижаются и спасения, и вредные замены. Пример выбран после просмотра пилота; отдельный статистический тест компонентов пока не проведён. Более широкий post-hoc анализ accuracy с общей поправкой на множественность не установил убедительного превосходства правил в проверенном режиме. Отсутствие значимости не означает практическую эквивалентность.

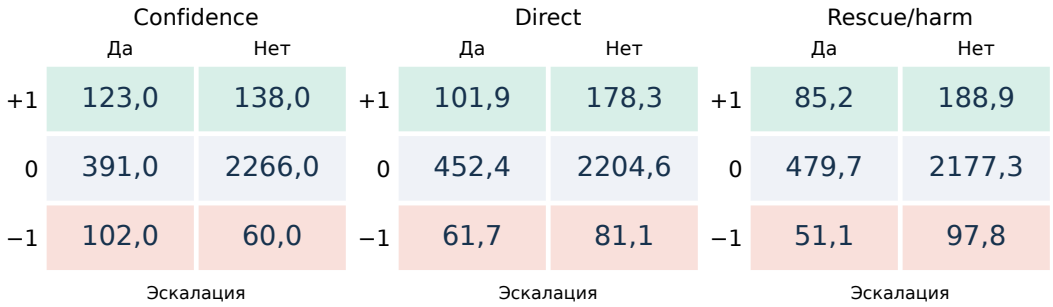


Рис. 2: TF–IDF → BERT-tiny, fit 1 000/dev 500, 20% вызовов, средние counts по 10 обучениям. Строки — выигрыш выбранного действия относительно противоположного; колонки — эскалация да/нет.

4.6. Воспроизводимость

Исходники и данные: https://github.com/Nastik05/Cost-sensitive-routing. Команда `python3 code/run.py` заново обучает роутеры по фиксированному кэшу, проверяет 1 440 строк метрик против исходного прогона и генерирует рисунки/таблицу. Это повтор обучения роутеров, а не независимое переобучение экспертов или новый test. Параметры экспертов, source revisions и хеши сохранены в `code/data/manifest.json`. Вычисления и визуализация разделены; численные фрагменты текста строятся из результатов автоматически.

Список литературы

[1] Wittawat Jitkrittum, Neha Gupta, Aditya Krishna Menon, Harikrishna Narasimhan, Ankit Singh Rawat, and Sanjiv Kumar. When does confidence-based cascade deferral suffice? In *Advances in Neural Information Processing Systems*, 2023. doi: 10.52202/075280-0431. URL https://proceedings.neurips.cc/paper_files/paper/2023/hash/1f09e1ee5035a4c3fe38a5681cae5815-Abstract-Conference.html.

[2] Harikrishna Narasimhan, Wittawat Jitkrittum, Aditya Krishna Menon, Ankit Singh Rawat, and Sanjiv Kumar. Post-hoc estimators for learning to defer to an expert. In *Advances in Neural Information Processing Systems*, 2022. doi: 10.52202/068431-2124. URL https://proceedings.neurips.cc/paper_files/paper/2022/hash/bc8f76d9caadd48f77025b1c889d2e2d-Abstract-Conference.html.

[3] Neha Gupta, Harikrishna Narasimhan, Wittawat Jitkrittum, Ankit Singh Rawat, Aditya Krishna Menon, and Sanjiv Kumar. Language model cascades: Token-level uncertainty and beyond. In *International Conference on Learning Representations*, 2024. URL https://proceedings.iclr.cc/paper_files/paper/2024/hash/11f5520daf9132775e8604e89f53925a-Abstract-Conference.html.

[4] Pranjal Aggarwal, Aman Madaan, Ankit Anand, Srividya Pranavi Potharaju, Swaroop Mishra, Pei Zhou, Aditya Gupta, Dheeraj Rajagopal, Karthik Kappaganthu, Yiming Yang, Shyam Upadhyay, Manaal Faruqui, and Mausam. AutoMix: Automatically mixing language models. In *Advances in Neural Information Processing Systems*,

2024. URL https://proceedings.neurips.cc/paper_files/paper/2024/hash/ecda225cb187b40ea8edc1f46b03ffda-Abstract-Conference.html.

[5] Isaac Ong, Amjad Almahairi, Vincent Wu, Wei-Lin Chiang, Tianhao Wu, Joseph E. Gonzalez, M Waleed Kadous, and Ion Stoica. RouteLLM: Learning to route LLMs with preference data. In *International Conference on Learning Representations*, 2025. URL https://proceedings.iclr.cc/paper_files/paper/2025/hash/5503a7c69d48a2f86fc00b3dc09de686-Abstract-Conference.html. Title follows the downloaded proceedings PDF; the proceedings metadata has a from/with wording discrepancy.

[6] Tianyi Niu, Justin Chen, Genta Indra Winata, Shi-Xiong Zhang, Supriyo Chakraborty, Sambit Sahu, Yue Zhang, Elias Stengel-Eskin, and Mohit Bansal. Routing with generated data: Annotation-free LLM skill estimation and expert selection. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 32441–32466, 2026. doi: 10.18653/v1/2026.acl-long.1498. URL https://aclanthology.org/2026.acl-long.1498/.

[7] Otto Nyberg and Arto Klami. Exploring uplift modeling with high class imbalance. *Data Mining and Knowledge Discovery*, 37:736–766, 2023. doi: 10.1007/s10618-023-00917-9. URL https://link.springer.com/article/10.1007/s10618-023-00917-9.

[8] Jasper Dekoninck, Maximilian Baader, and Martin Vechev. A unified approach to routing and cascading for LLMs. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267, pages 12987–13010. PMLR, 2025. URL https://proceedings.mlr.press/v267/dekoninck25a.html.

[9] Krzysztof Rudaś and Szymon Jaroszewicz. Linear regression for uplift modeling. *Data Mining and Knowledge Discovery*, 32:1275–1305, 2018. doi: 10.1007/s10618-018-0576-8. URL https://link.springer.com/article/10.1007/s10618-018-0576-8.

[10] Sören R. Künzel, Jasjeet S. Sekhon, Peter J. Bickel, and Bin Yu. Metalearners for estimating heterogeneous treatment effects using machine learning. *Proceedings of the National Academy of Sciences*, 116(10):4156–4165, 2019. doi: 10.1073/pnas.1804597116. URL https://doi.org/10.1073/pnas.1804597116.

[11] Hao Li, Yiqun Zhang, Zhaoyan Guo, Chenxu Wang, Shengji Tang, Qiaosheng Zhang, Yang Chen, Biqing Qi, Peng Ye, Lei Bai, Zhen Wang, and Shuyue Hu. LLMRouterBench: A massive benchmark and unified framework for LLM routing. In *Findings of the Association for Computational Linguistics: ACL 2026*, pages 37733–37754, 2026. doi: 10.18653/v1/2026.findings-acl.1881. URL https://aclanthology.org/2026.findings-acl.1881/.

[12] Divyat Mahajan, Ioannis Mitliagkas, Brady Neal, and Vasilis Syrgkanis. Empirical analysis of model selection for heterogeneous causal effect estimation. In *International Conference on Learning Representations*, 2024. URL https://openreview.net/forum?id=yuy6cGt3KL. Spotlight; full text inspected through arXiv:2211.01939v3.

[13] Iñigo Casanueva, Tadas Temčinas, Daniela Gerz, Matthew Henderson, and Ivan Vulić. Efficient intent detection with dual sentence encoders. In Tsung-Hsien Wen, Asli Celikyilmaz, Zhou Yu, Alexandros Papangelis, Mihail Eric, Anuj Kumar, Iñigo Casanueva, and Rushin Shah, editors, *Proceedings of the 2nd Workshop on Natural Language Processing for Conversational AI*, pages 38–45, Online, July 2020. Association for Computational Linguistics. doi: 10.18653/v1/2020.nlp4convai-1.5. URL https://aclanthology.org/2020.nlp4convai-1.5/.